

Transformer

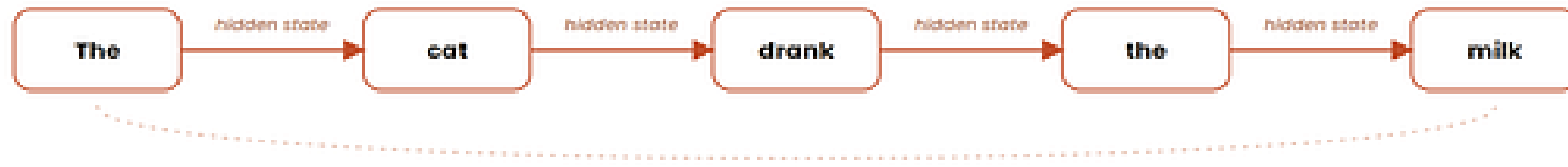
The modern AI architecture

Dr. Grantham Pang
The University of Hong Kong



**RNN / LSTM — sequential processing**

Each token only sees a summary passed forward from the one before it.

RNN/LSTM : Sequential

Information from "The" must travel through every step to reach "milk" — it can fade along the way.

**Self-attention — direct, all at once**

Every token can look directly at every other token, regardless of distance.

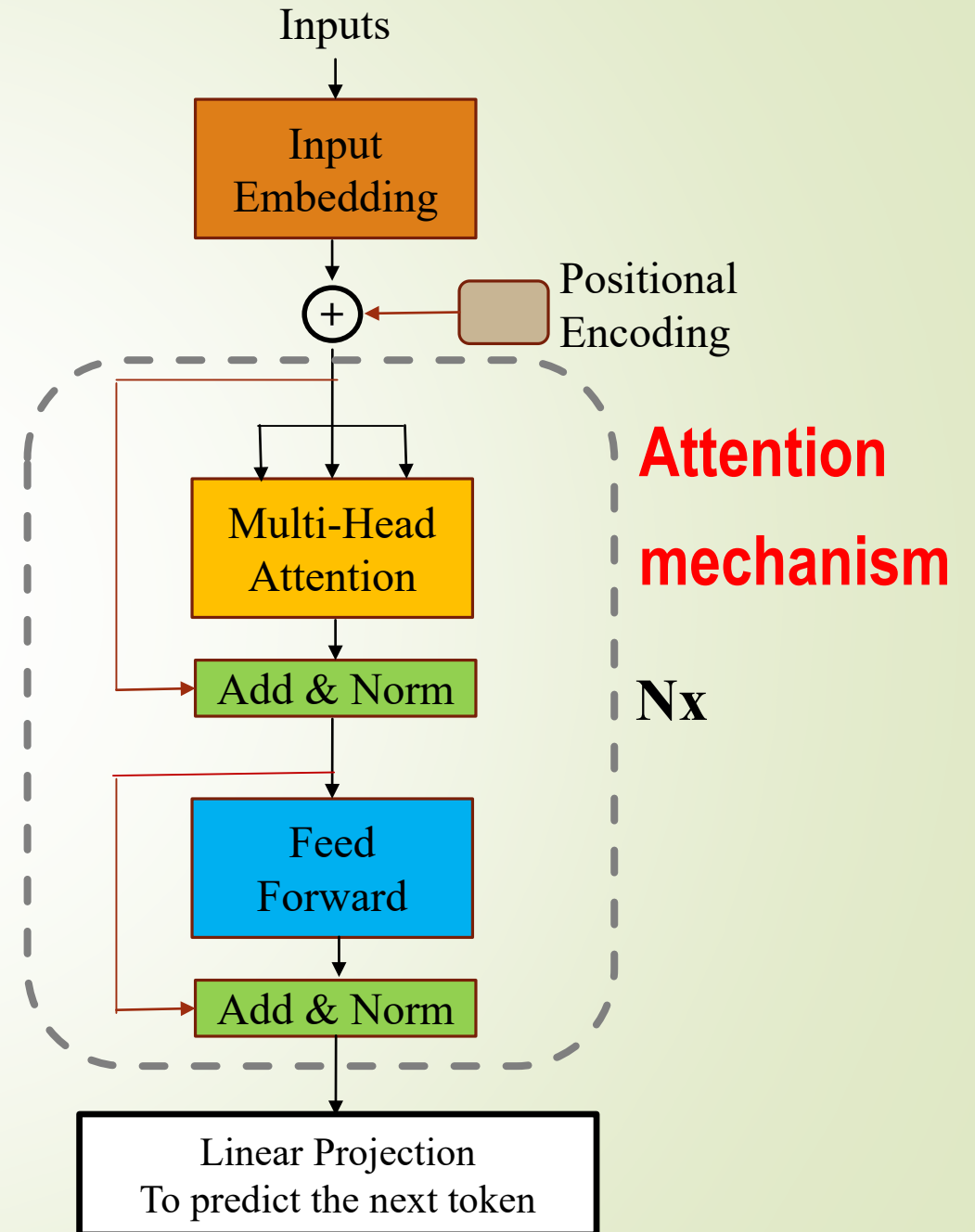
Self-attention : All at once, parallel

A Transformer can process the relationships between many tokens at the same time.

"milk" can attend directly back to "The" — no chain to travel through

This direct, all-pairs comparison is exactly what self-attention computes with QK^T.

Attention



Attention is all you need !

A Transformer can process the relationships between many tokens at the same time — making it much easier to model relationships between words that may be far apart in a sentence.

It asks: “Which tokens are important for understanding me?”

It then gives more weight to the tokens that are more relevant and less weight to the tokens that are less relevant.

The cat drank the milk

Attention uses the Query and Key to decide how relevant each token is, and uses those weights to combine the Values.

2. The Basic

Input embeddings



You can think of embeddings as coordinates in a high-dimensional mathematical space.

The model can learn useful relationships between representations in this space.

Token



Embedding



[0.21, -0.73, 0.14, ...]



Positional Encoding: Teaching the Model About Word Order

“Dog bites man.” and “Man bites dog.” are different.

The same words appear, but their order changes the meaning.

Transformers therefore need a way to represent positional information. The original Transformer used sinusoidal positional encodings.



Imagine that every token goes to a librarian's desk. Each token brings three things:

1.Query — “*What information am I looking for?*” The Query represents what this token is looking for in the other tokens.

2.Key — “*What information do I have?*” The Key describes what a token can offer to other tokens.

3.Value — “*Here is my actual information.*” The Value contains the information that the token can provide.

This is the basic idea behind self-attention.

Now imagine that one token wants to understand its context. It takes its **Query** and **compares it with the Key of every other token.**



My Query



Compare with everyone's Keys



How relevant is each token?



Assign attention weights



Combine the Values



New representation

How the Query, Key, and Value vectors are created and compared ?



But before that, we need positional encoding.

The model needs to know **where** each token appears in the sentence.

Self-attention compares tokens with one another, but we also need to give the model information about their positions.

Tokens



Embeddings



Add positional information



Queries, Keys, Values



Attention scores



Attention weights



Weighted combination of Values



Attention output

Input Token embeddings

To represent each token as a vector of numbers.

"cat" $\rightarrow$ [0.21, -0.43, 0.87, ...]

If our sentence contains n tokens, we get an embedding vectors. We can place them together into a matrix.

$$z_i = x_i \text{ (input embeddings)} \\ + PE_i \text{ (positional encoding)}$$

Step 2 – add positional encoding to the token embedding

$x_1 =$

0

1

0

1

+

$PE_1 =$

.8415

.5403

.0100

1.000

=

$z_1 =$

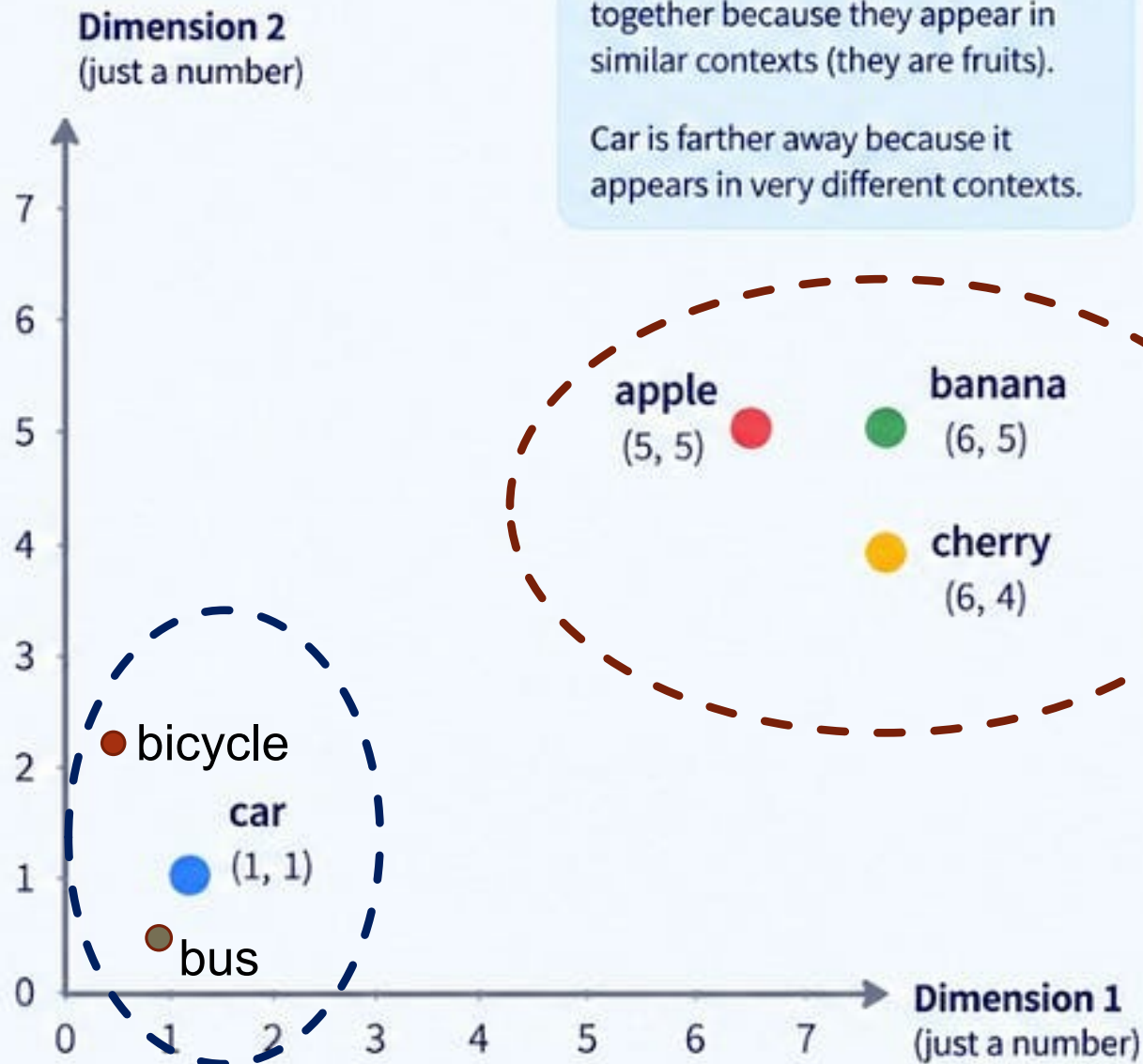
.8415

1.540

.0100

2.000

A Simple 2D Example



Semantically similar words are close together in the 2-D embedding space.

Each word (concept) is represented by a point in a high dimensional space.

3. The Core: Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

QK^T



Calculate relevance scores

$\div \sqrt{d_k}$



Scale the scores

softmax



Convert scores into attention weights

$\times V$



Create a weighted combination of Values

Main Idea:
Attention uses the Query and Key to decide how relevant each token is, and then uses those weights to combine the Values.

Multi-head attention

A Transformer does not usually calculate attention only once. Instead, it uses several separate attention heads. Each head can learn to focus on different relationships between tokens.

$$\text{head}_j = \text{Attention}(Q_j, K_j, V_j)$$

$$\text{Concat}(\text{head}_1, \dots, \text{head}_h)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_o$$

h = number of attention heads

W_o = learned output projection



↓
Concatenate
↓
Output projection
↓
Output

Different weight matrices are learned from training.

Input representations

↓
Create **Q**, K, V
↓
Compare **Q** with K
↓
Scale the scores
↓
Apply softmax
↓
Get attention weights
↓
Use weights **to** combine V
↓
Attention output

Example: Stock price prediction

A transformer time series forecasting example.

It uses the previous 20 days of stock price of a stock to predict

the next value. The daily closing price of the previous 20

trading days is used as input to predict the next closing price.

$$\underbrace{x_{t-19} \quad x_{t-18} \quad \dots \quad x_t}_{\text{Input sequence}} \longrightarrow \text{Predict next value}$$

How do these values relate to each other to give a prediction of the next value ?

Lookback = 20 (input sequence length = 20)

D_model = 64 (64-dim vector for each time step)

N_head = 4 (4 ways to look at the relationships)

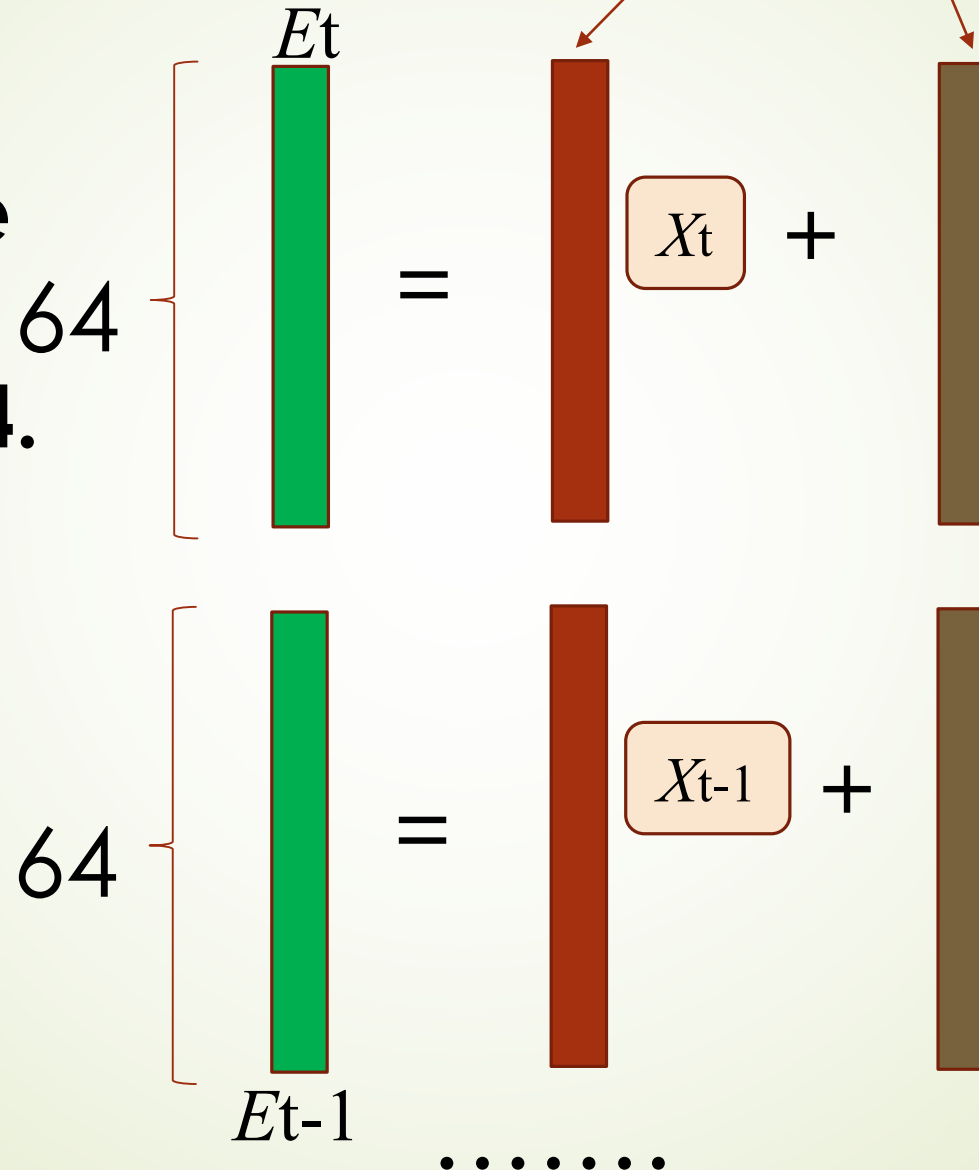
N_layers = 2 (two attention blocks)

parameters [64+64=128]

x_{t-19} x_{t-18} ... x_t

$\longrightarrow$ Predict next value

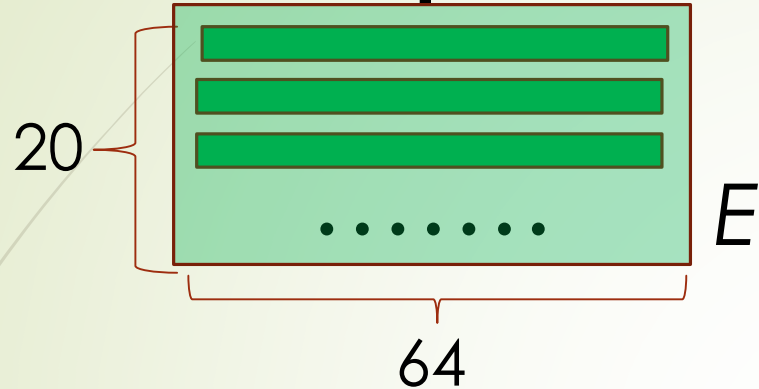
Input Projection:
Each single value
is projected to
a dimension of 64.



Each time step x_t is projected to a hidden dimension d_{model} via a linear layer.

Input Projection:

Hence, input becomes (20x64)



$$\begin{array}{c} \text{Input} \\ X \end{array} = \begin{array}{c} \text{input projection} \\ E \end{array} + \begin{array}{c} \text{positional encoding} \\ P \end{array}$$



Attention mechanism (matching Q and K) are carried out in a lower dimension than the original 64-dim (D_{MODEL}).

Each attention head has dimension 16:

$$d_k = \frac{D_{\text{MODEL}}}{N_{\text{HEAD}}} = \frac{64}{4} = 16$$

$$\underset{(20 \times 64)}{Q} = \underset{(20 \times 64)}{X} \underset{(64 \times 64)}{W_q} + b_q$$

To be exact:

The number of parameters are $(64 \times 64) + (1 \times 64)$

W_q , W_k , W_v , W_o are the weight parameters,
All are dimension (64×64) .

$$K = X W_k + b_k$$

$$\underset{(20 \times 64)}{V} = \underset{(20 \times 64)}{X} \underset{(64 \times 64)}{W_v} + b_v$$

The matrices Q, K, V are partitioned into 4 sections (as we will use 4 heads) in the transformer block.

$$Q = \begin{bmatrix} \textcolor{red}{Q1} & | & Q2 & | & Q3 & | & Q4 \end{bmatrix}$$

(20x64) (20x16) each

$$K = \begin{bmatrix} \textcolor{red}{K1} & | & K2 & | & K3 & | & K4 \end{bmatrix}$$

(20x16) each

$$V = \begin{bmatrix} \textcolor{red}{V1} & | & V2 & | & V3 & | & V4 \end{bmatrix}$$

(20x16) each

Dimension of $Q_1 \cdot K_1^T$ is
 $(20 \times 16) (16 \times 20) = (20 \times 20)$

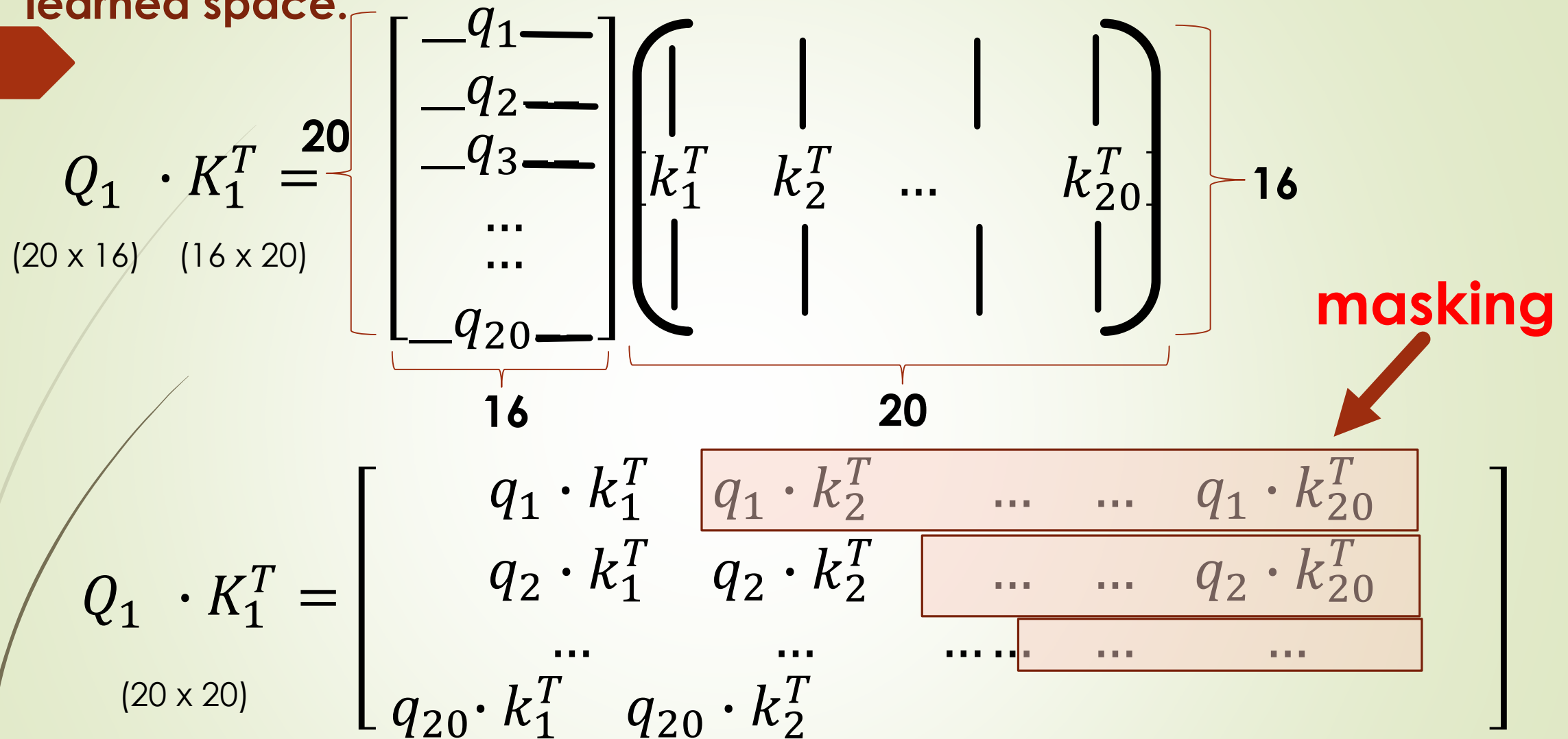
Head 1:

$$\text{score} = \frac{\textcolor{red}{Q_1} \cdot \textcolor{red}{K_1^T}}{\sqrt{16}} \quad \text{Dimension of score is } (20 \times 20)$$

How relevant is one value to another value in the input sequence of 20 values ?

Answer: This relevance measure/score is carried out by dot-product of Q1 and K1 transpose. This matching is carried out in a dimension of 16.

Details : a large dot-product means how the two vectors are more aligned in the learned space.



$$\begin{array}{c}
 Q_1 \cdot K_1^T = \left[\begin{array}{c} -q_1- \\ -q_2- \\ -q_3- \\ \vdots \\ -q_{20}- \end{array} \right] \left(\begin{array}{c} | \\ | \\ | \\ \vdots \\ | \end{array} \right) \left[\begin{array}{c} k_1^T \\ k_2^T \\ \dots \\ k_{20}^T \end{array} \right] \quad \begin{array}{c} 20 \\ 16 \end{array}
 \end{array}$$

$\underbrace{\hspace{10em}}_{16} \quad \underbrace{\hspace{10em}}_{20}$

$$Q_1 \cdot K_1^T = \left[\begin{array}{ccccc} q_1 \cdot k_1^T & q_1 \cdot k_2^T & \dots & \dots & q_1 \cdot k_{20}^T \\ q_2 \cdot k_1^T & q_2 \cdot k_2^T & \dots & \dots & q_2 \cdot k_{20}^T \\ \vdots & \vdots & \dots & \dots & \vdots \\ q_{20} \cdot k_1^T & q_{20} \cdot k_2^T & \dots & \dots & q_{20} \cdot k_{20}^T \end{array} \right]$$

(20×20)

(attention matrix) of head1 is (20 x 20) and the first row is how the others response to the first query. Softmax of each row is needed, and so the sum of each row is 1.



Each number tells us how strongly one token relates to another before normalization. These are called the **raw attention scores**.

- **The dot product measures alignment.**
 - **If two vectors point in similar directions, the dot product is large and positive.**
 - **If they are unrelated, it is near zero.**
 - **If they are opposites, it is negative.**
- 

Why divide by $\sqrt{d_k}$?

Our raw scores can become very large when the Query and Key vectors have high dimensions. Why is that a problem?

It is because the next step is **softmax**.

If the scores become extremely large, softmax can become too confident: Examples.

Ex 1: [2.1, 1.8, 1.2] is raw score

↓

softmax

↓

[0.48, 0.36, 0.16] is result

Ex 2: [20, 2, 1] is large raw score

↓

softmax

↓

[almost 1, almost 0, almost 0]

To keep the scores in a more manageable range, we divide them by $\sqrt{d_k}$

score = $\frac{Q_1 \cdot K_1^T}{\sqrt{16}}$ Dimension of attention score is (20x20)

This is called **scaling**. The idea is simple: ***As the size of the Query and Key vectors grows, divide the dot-product scores by $\sqrt{d_k}$ to keep their scale under control.***

Next is to turn the scores into weights.

Example of scaled attention scores.

E.g.: [2.1, 0.7, -0.4]

They are not convenient as attention weights. We want numbers that:

- are between 0 and 1
- add up to 1

This is exactly what softmax gives us.

The softmax formula is:

$$\alpha_{ij} = e^{s_{ij}} / \sum_j e^{s_{ij}}$$

where s_{ij} is the scaled attention score between token i and token j .

e.g. Attention scores of one row in score matrix is

[2.1, 0.7, -0.4]

After softmax:

Attention weights = **softmax**(scores)

Attention weights: [0.76, 0.19, 0.05] **sum of the row = 1**

Our stock value prediction example:

Head1 = Attention weights · **[V1]** **is output from Head1**
(20x16) (20x20) (20x16)

Head1 contains the value of one learned relationship through the attention mechanism.

Attention weights = $\text{softmax}(\text{scores}) = \text{softmax}(Q_1 K_1^T / \sqrt{d_k})$
(20x20) (20x20)

$$\text{softmax}(Q_1 \cdot K_1^T) = \begin{bmatrix} \alpha_1 & \alpha_2 & \dots & \dots & \alpha_{20} \\ \beta_1 & \beta_2 & \dots & \dots & \beta_{20} \\ \dots & \dots & \dots & \dots & \dots \\ \gamma_1 & \gamma_2 & \dots & \dots & \gamma_{20} \end{bmatrix}$$

Attention weight matrix (20 x 20) has the first row, which is how the key of other token response to the query from the first token.

Softmax of the matching score of each row is needed, and so the sum of each row is 1.

Next, we will examine getting the **Value** from the scaled dot-product attention weight matrix

$$\text{softmax} \left[\begin{matrix} Q_1 K_1^T / \sqrt{d_k} \\ (20 \times 20) \end{matrix} \right] \left[\begin{matrix} V_1 \\ (20 \times 16) \end{matrix} \right]$$

Value from the scaled dot-product attention weight

$$\text{Head}_1 = \underset{(20 \times 20)}{\text{softmax} [Q_1 K_1^T / \sqrt{d_k}]} \underset{(20 \times 16)}{[V_1]} \quad \begin{array}{l} (1 \times 16) \text{ vector, value for the} \\ \text{query of row 1 of } Q_1 \end{array}$$

$$= \underset{20}{\begin{bmatrix} \alpha_1 & \alpha_2 & \dots & \dots & \alpha_{20} \\ \beta_1 & \beta_2 & \dots & \dots & \beta_{20} \\ \dots & \dots & \dots & \dots & \dots \\ \gamma_1 & \gamma_2 & \dots & \dots & \gamma_{20} \end{bmatrix}} \cdot \underset{[V_1]}{\begin{bmatrix} v_1^1 \\ v_1^2 \\ \dots \\ v_1^{20} \end{bmatrix}}$$

$$= \underset{20}{\begin{bmatrix} \alpha_1 v_1^1 + \alpha_2 v_1^2 + \alpha_3 v_1^3 + \dots + \alpha_{20} v_1^{20} \\ \gamma_1 v_1^1 + \gamma_2 v_1^2 + \gamma_3 v_1^3 + \dots + \gamma_{20} v_1^{20} \end{bmatrix}}$$

$$\underset{(20 \times 20)}{\text{softmax} [Q_1 K_1^T / \sqrt{d_k}]}$$

$$= \underset{20}{\begin{bmatrix} \alpha_1 & \alpha_2 & \dots & \dots & \alpha_{20} \\ \beta_1 & \beta_2 & \dots & \dots & \beta_{20} \\ \dots & \dots & \dots & \dots & \dots \\ \gamma_1 & \gamma_2 & \dots & \dots & \gamma_{20} \end{bmatrix}} \cdot \underset{16}{\begin{bmatrix} v_1^1 \\ v_1^2 \\ \dots \\ v_1^{20} \end{bmatrix}}$$

$$= \underset{20}{\begin{bmatrix} \alpha_1 v_1^1 + \alpha_2 v_1^2 + \alpha_3 v_1^3 + \dots + \alpha_{20} v_1^{20} \\ \gamma_1 v_1^1 + \gamma_2 v_1^2 + \gamma_3 v_1^3 + \dots + \gamma_{20} v_1^{20} \end{bmatrix}}$$

20

16

16

$$\text{Head}_1 = \underset{(20 \times 16)}{\text{softmax}} \left[\underset{(20 \times 20)}{Q_1 K_1^T} / \sqrt{d_k} \right] \underset{(20 \times 16)}{[V_1]}$$

Head₁ is a **weighted combination of the Value vectors**. The tokens that received larger attention weights will have more value to contribute to the output.

Yet, Transformers usually use several attention heads. It is because a sentence (or a number sequence) can contain many different kinds of relationships at the same time.

Head₁ has its own Query, Key, and Value projection matrices (Q_1, K_1, V_1):

$$\underset{(20 \times 16)}{\text{Head}_1} = \underset{(20 \times 20)}{\text{softmax}} \left[\underset{(20 \times 20)}{Q_1 K_1^T} / \sqrt{d_k} \right] \underset{(20 \times 16)}{[V_1]}$$

Scaled Dot-Product Attention

A single attention operation gives you one way of looking at the sentence. One attention head cannot capture all of these at once. **Solution: Multi-Head Attention.**

Instead of one set of W_Q, W_K, W_V use h sets.

Each head learns a different “**perspective**”:

Head 1 might learn **co-reference** (“it” → “cat”)

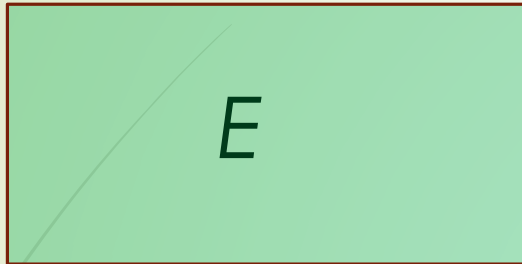
Head 2 might learn **syntax** (“sat” → “on”)

Head 3 might learn **tense relationships**

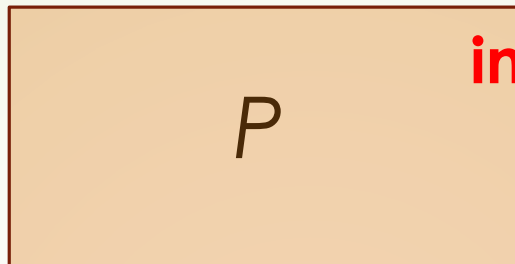
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_h)W^O$$

To summarize: $x_{t-19} \ x_{t-18} \ \dots \ x_t \longrightarrow$ Predict next value

input projection



positional encoding

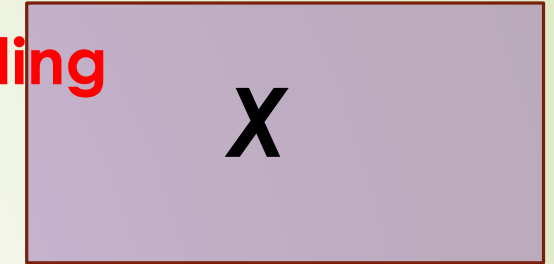


+

input embedding

gives

Bring to 64-dim



(20x64)

$$Q = X W_q + b_q$$

(20x64)

$$Q = [\text{Q1} \mid Q2 \mid Q3 \mid Q4]$$

(20x64) (20x16) each

$$K = X W_k + b_k$$

$$K = [\text{K1} \mid K2 \mid K3 \mid K4]$$

(20x16) each

$$V = X W_v + b_v$$

$$V = [\text{V1} \mid V2 \mid V3 \mid V4]$$

(20x16) each

Creation of the Query, Key and Value matrices, which will be used or the Attention mechanism in Transformer. If number of Head is 4, Q, K, V will be divided into 4 partitions.

$$\text{Head}_1 = \underset{(20 \times 16)}{\text{softmax}} \left[\underset{(20 \times 20)}{Q_1 K_1^T} / \sqrt{d_k} \right] \underset{(20 \times 16)}{[V_1]}$$

$$\text{Head}_2 = \underset{(20 \times 16)}{\text{softmax}} \left[\underset{(20 \times 20)}{Q_2 K_2^T} / \sqrt{d_k} \right] \underset{(20 \times 16)}{[V_2]}$$

$$\text{Head}_3 = \underset{(20 \times 16)}{\text{softmax}} \left[\underset{(20 \times 20)}{Q_3 K_3^T} / \sqrt{d_k} \right] \underset{(20 \times 16)}{[V_3]}$$

$$\text{Head}_4 = \underset{(20 \times 16)}{\text{softmax}} \left[\underset{(20 \times 20)}{Q_4 K_4^T} / \sqrt{d_k} \right] \underset{(20 \times 16)}{[V_4]}$$

Scaled dot-product to find relevance
among the input sequence.

Matching in 16-dim

Attention scores are changed to
attention weights by softmax()

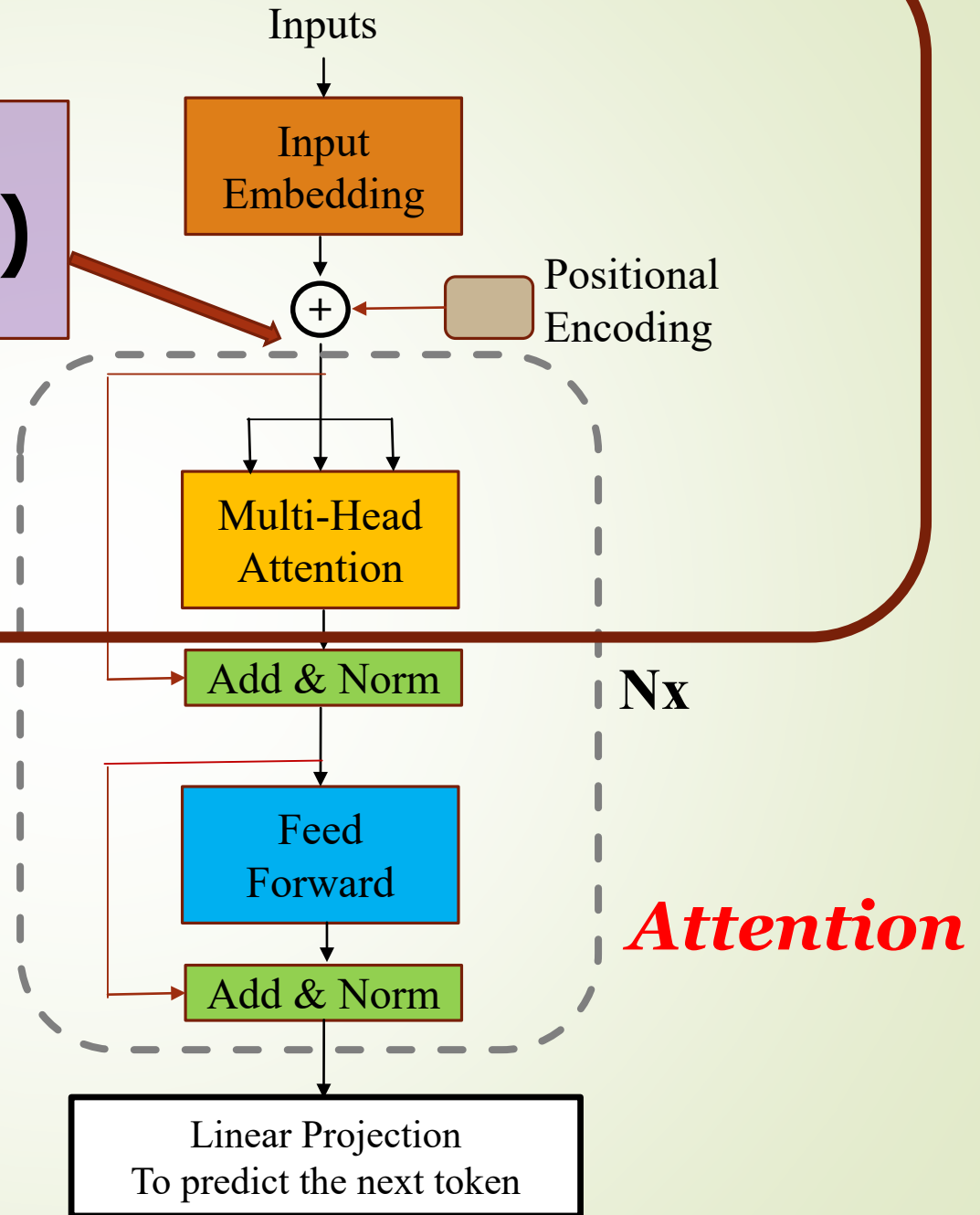
Extract the actual Information by multiplying with the Value vectors

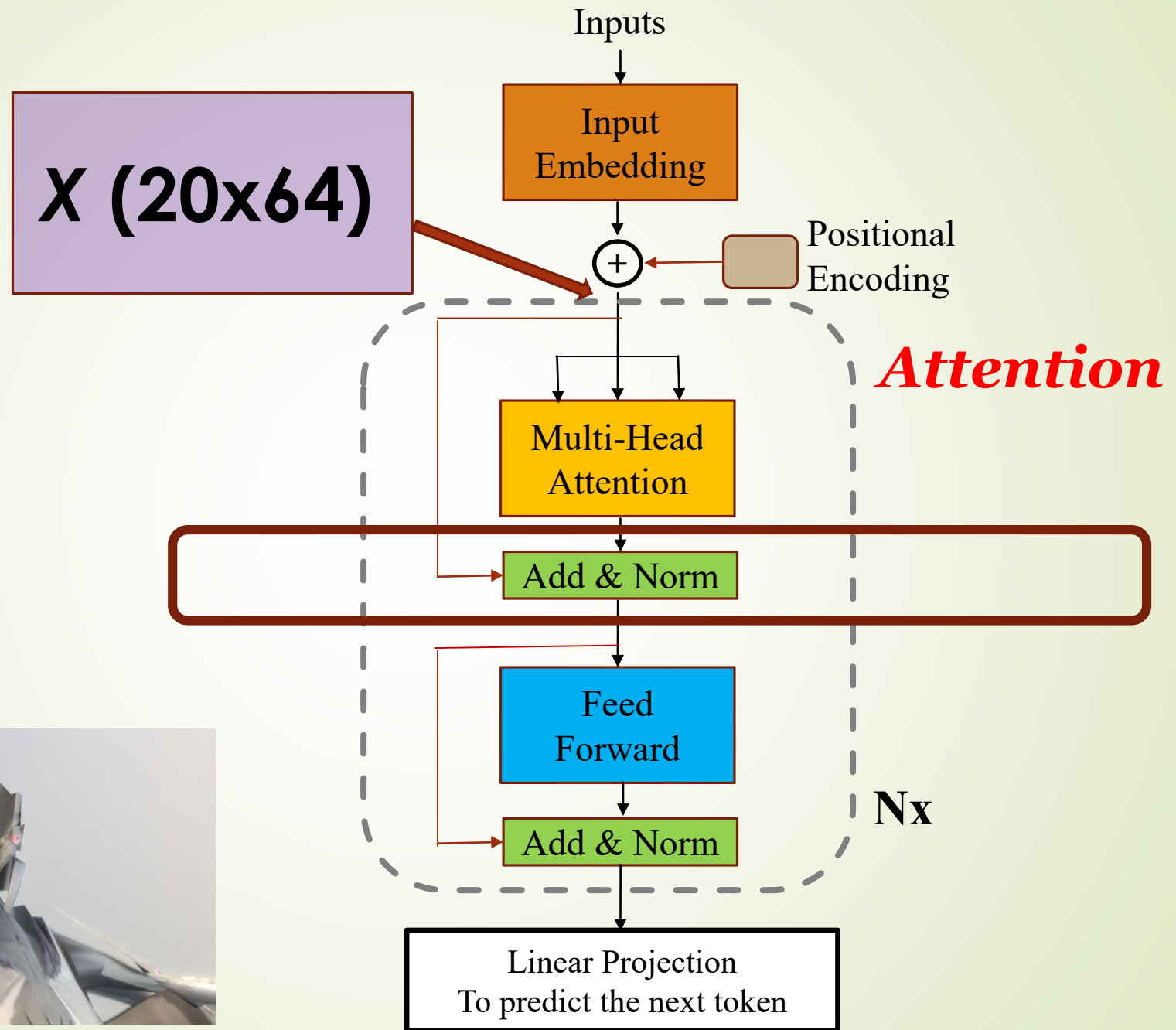
$$\begin{aligned} \text{Output} &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_o \\ \underset{(20 \times 64)}{} &= \underset{(20 \times 64)}{\begin{bmatrix} \text{Head1} & \text{Head2} & \text{Head3} & \text{Head4} \end{bmatrix}} \underset{(64 \times 64)}{[W_o]} + b_o \end{aligned}$$

**Multi-head
attention**



$X (20 \times 64)$





Add and Normalize: Residual connections and layer normalization

- **Residual connections** add each sub-layer's input back to its output. This gives the network a “shortcut path” — information can skip past a layer entirely if that layer isn't useful for a given token, which prevents important signal from getting lost or distorted across many layers.

$$\begin{aligned}h_{\text{tmp}} &= X + \text{Attention}(X) \\ &= X + \text{Output}\end{aligned}$$

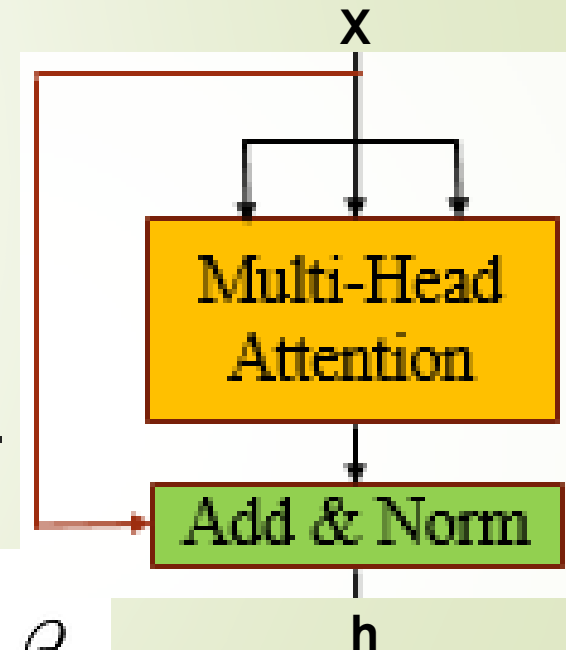
- **Layer normalization** rescales the numbers flowing through the network, keep values in a stable range so training does not blow up or vanish as depth increases.

$$h = \text{LayerNorm}(h_{\text{tmp}})$$

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

128 [64+64] parameters

Two learned parameters



The feed-forward network

After attention gathers context from other tokens, each token's representation passes through a feed-forward network — a small, fully-connected neural network applied identically to every token position.

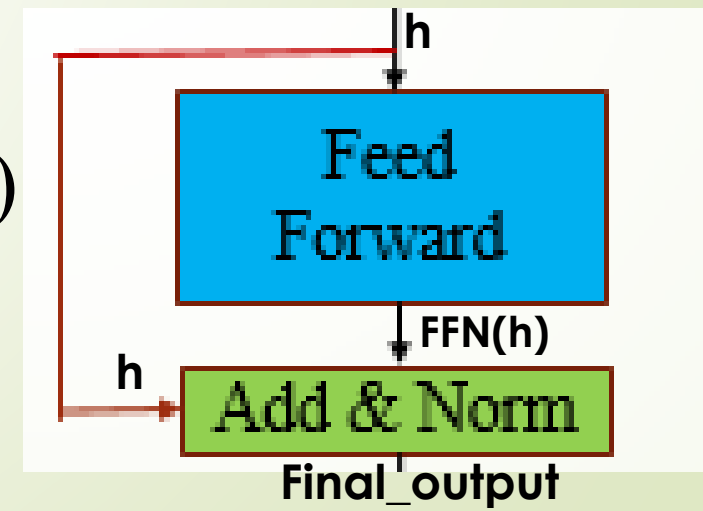
If attention is about *gathering* relevant information from across the sequence, the feed-forward layer is about *processing* that information — transforming it non-linearly to extract higher-level features.

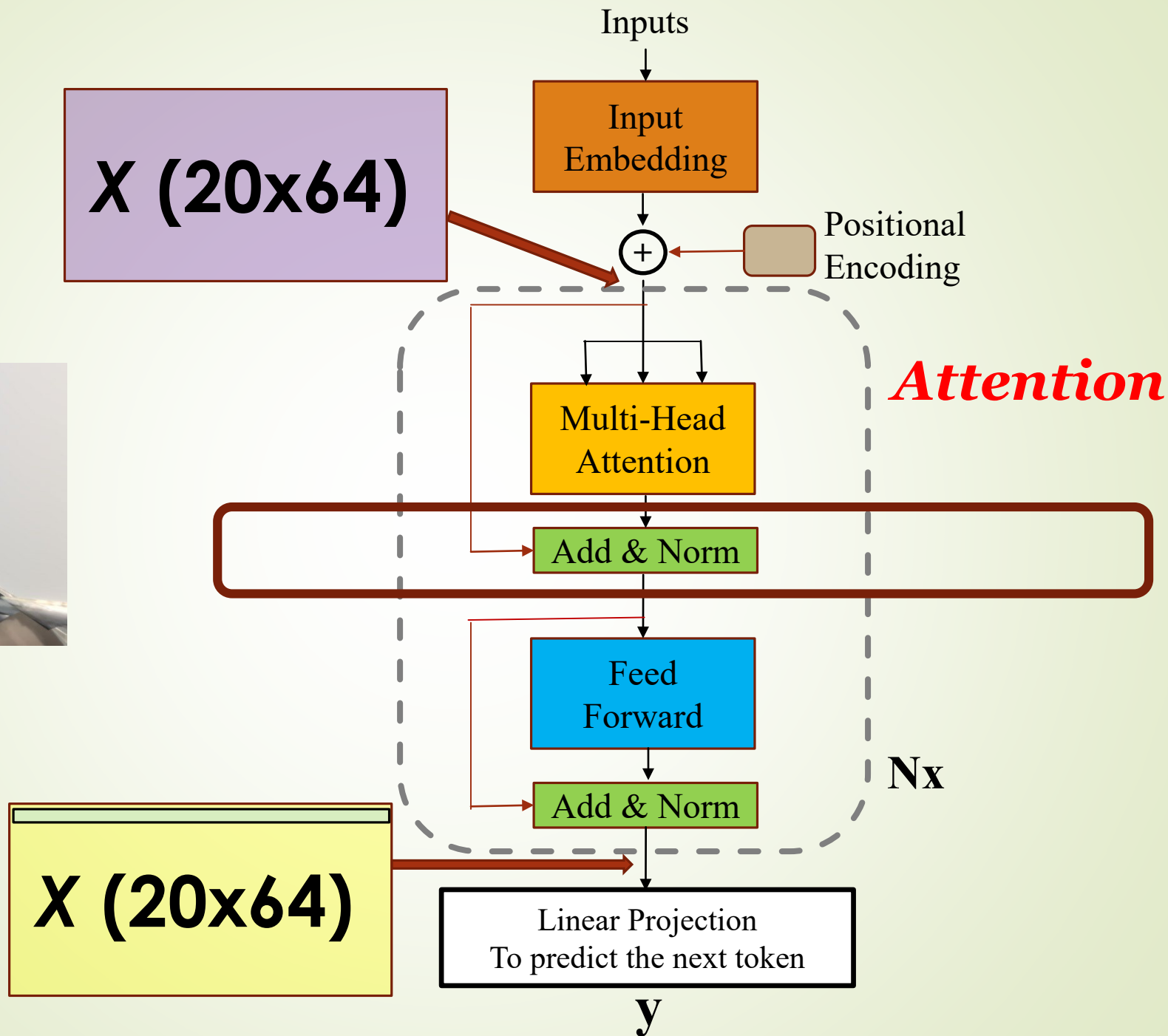
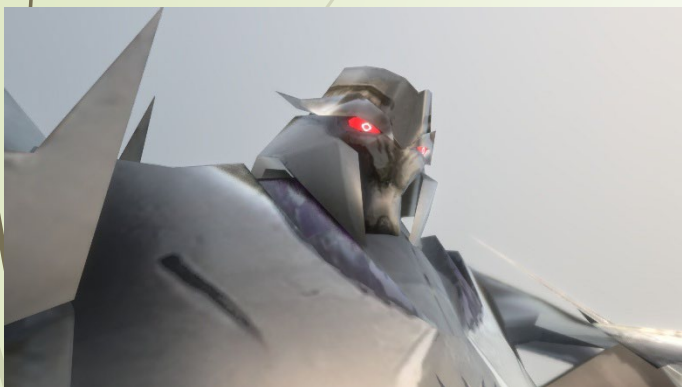
$$\text{Final_output} = \text{LayerNorm}(h + \text{FFN}(h))$$

FFN parameters:

$$64 \rightarrow 128 \quad \# \text{parameters} = 64 \times 128 + 128 = 8320$$

$$128 \rightarrow 64 \quad \# \text{parameters} = 128 \times 64 + 64 = 8256$$





The Final Linear Projection (after multiple Attention blocks)

Take the final time-step of 64-dim vector, carry out a linear projection to get the predicted value:

X (20x64)



Predict next value

X (20x64)

← Final time-step of 64-dim vector

$$y(\text{ValuePred}) = X_{t_{\text{attention}}} \cdot W_{\text{out}} + b_{\text{out}}$$

(1×64) (64×1) (1×1)

$$\# \text{parameters} = 64 + 1 = 65$$

Prediction

Cross-Entropy Loss

During training, the model knows what the correct next token should be (it's in the training data). **Cross-entropy loss** measures how wrong the model's predicted probability distribution is compared to the actual answer.

The goal of training: minimize this loss across the entire training dataset.

Backpropagation

Once the loss is computed, **backpropagation** traces the error backward through the network — computing how much each parameter contributed to the error. This is done using the chain rule of calculus, calculating gradients for every one of the model's billions of parameters.

Real-world analogy: Imagine a factory where a defective product is traced back through every step of assembly to find exactly which worker made which mistake, and by how much. Backpropagation does this for every parameter in the model, simultaneously.

Optimizer (Adam, SGD, etc.)

Once we know the gradients, an **optimizer** uses them to update the model's parameters in a direction that reduces the loss.

- **SGD (Stochastic Gradient Descent)**: The original optimizer — simple but slow to converge
- **Adam (Adaptive Moment Estimation)**: The most common modern optimizer. Adapts the learning rate for each parameter individually, converging much faster and more stably.

This entire cycle — forward pass → loss → backprop → optimizer update — is repeated **trillions of times** across the training dataset, over weeks or months of compute time on thousands of GPUs.

Example: Training GPT-4 is estimated to have required thousands of A100 GPUs running for months, consuming compute resources valued at tens or hundreds of millions of dollars.

4. Example of Scaled dot-product attention

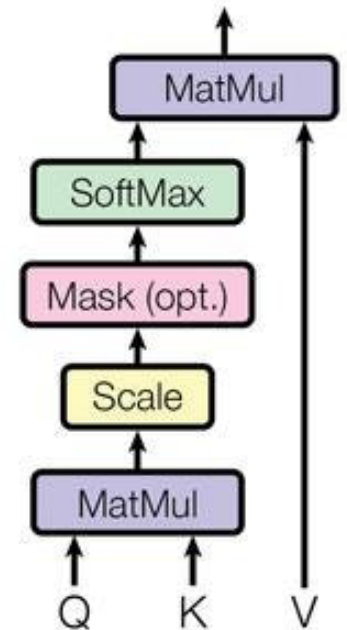
Every word's embedding vector x is projected into Q , K , and V using three learned weight matrices:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

Attention is computed with this single, famous formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention



Full numerical example of “Masked” attention

In masked attention, future tokens are purposefully nullified, so your predictions and scores only depend on the past tokens, not the coming ones. It uses a neat matrix trick called upper triangular mask;

An example: professor graded essays

$$Q = XW^Q, \quad (3 \times 9) = (3 \times 36)(36 \times 9)$$

$$K = XW^K, \quad (3 \times 9) = (3 \times 36)(36 \times 9)$$

$$V = XW^V, \quad (3 \times 9) = (3 \times 36)(36 \times 9)$$

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + \text{causal mask} \right) V.$$

Causal masking replaces scores corresponding to future positions with $-\infty$ before softmax, so their attention weights become zero.

$$d_k = \frac{D_{\text{MODEL}}}{N_{\text{HEAD}}}$$

$$= \frac{36}{4} = 9$$

Matching between Query and Key is carried out in 9-dim. Original dim is 36.

Examples of Q, K and V.

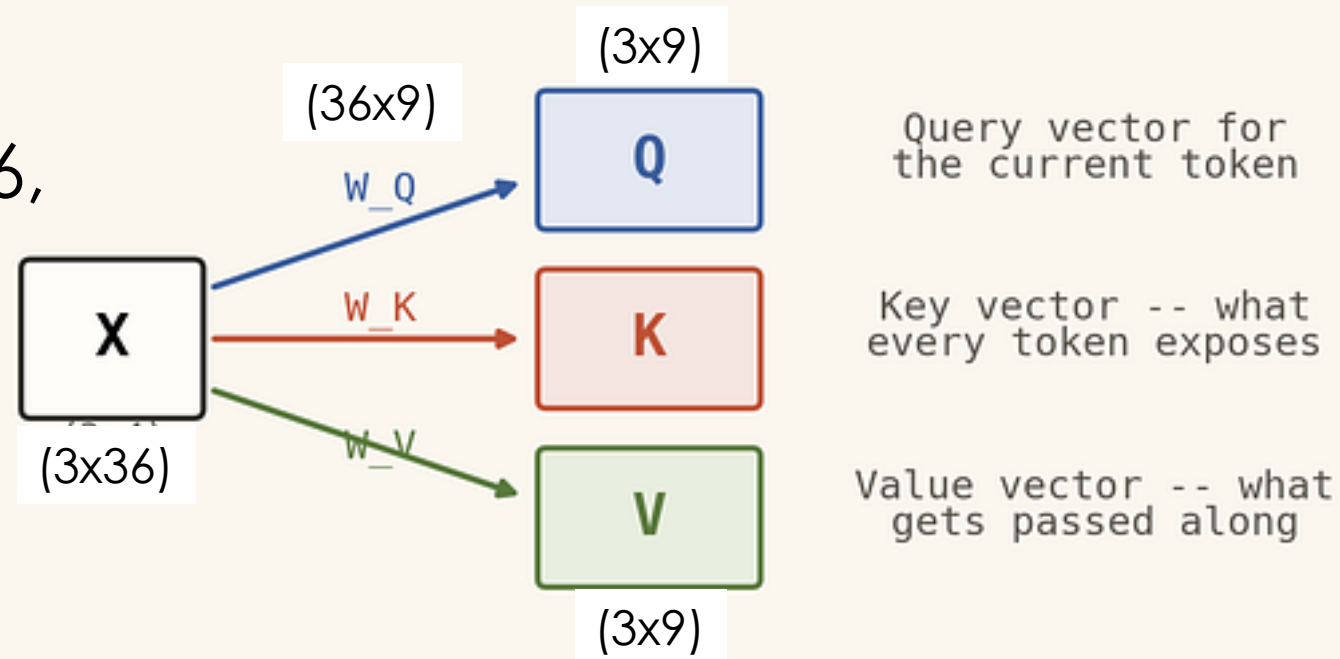
$$Q = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix},$$

$$K = \begin{bmatrix} 2 & 1 & 3 & 1 & 2 & 1 & 3 & 2 & 1 \\ 1 & 3 & 2 & 2 & 1 & 3 & 1 & 2 & 3 \\ 3 & 1 & 4 & 2 & 3 & 1 & 2 & 3 & 1 \end{bmatrix},$$

$$V = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ 10 & 20 & 30 & 40 & 50 & 60 & 70 & 80 & 90 \\ 100 & 200 & 300 & 400 & 500 & 600 & 700 & 800 & 900 \end{bmatrix}$$

An example: professor graded essays

Three tokens,
original
dimension = 36,
4 heads,
dimension for
matching = 9.
Q, K and V are
3×9 matrices.



Compute the raw attention-score matrix

The raw score matrix is

$$S = QK^T.$$

Using the specified Q and K ,

$$S = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 3 & 1 \\ 3 & 2 & 4 \end{bmatrix}$$

For example,

$$S_{1,3} = q_1 k_3^T = 1(3) + 0(1) + 0(4) + \cdots + 0(1) = 3.$$

Similarly,

$$S_{3,2} = q_3 k_2^T = 0(1) + 0(3) + 1(2) + \cdots + 0(3) = 2.$$

Scale the score matrix

The key/query embedding dimension is

$$d_k = 9.$$

Therefore,

$$\sqrt{d_k} = \sqrt{9} = 3.$$

Divide every element of S by 3:

$$S_{\text{scaled}} = \frac{S}{3}.$$

Thus,

$$S_{\text{scaled}} = \begin{bmatrix} 0.666667 & 0.333333 & 1.000000 \\ 0.333333 & 1.000000 & 0.333333 \\ 1.000000 & 0.666667 & 1.333333 \end{bmatrix}$$

Apply the causal mask

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}.$$

The masked score matrix is

$$S_{\text{masked}} = S_{\text{scaled}} + M.$$

Hence,

Becomes upper triangular after masking

$$S_{\text{masked}} = \begin{bmatrix} 0.666667 & -\infty & -\infty \\ 0.333333 & 1.000000 & -\infty \\ 1.000000 & 0.666667 & 1.333333 \end{bmatrix}$$

The $-\infty$ entries are not ordinary finite numbers; they indicate that those future positions must receive zero attention after softmax.

Apply row-wise softmax

Row 1

$$\text{softmax} \left(\begin{bmatrix} 0.666667 & -\infty & -\infty \end{bmatrix} \right) = \boxed{\begin{bmatrix} 1 & 0 & 0 \end{bmatrix}}$$

row 1 values

Position 1 can attend only to itself.

Row 2

$$\text{softmax} \left(\begin{bmatrix} 0.333333 & 1.000000 & -\infty \end{bmatrix} \right).$$

The finite exponentials are

$$e^{0.333333} \approx 1.395612, \quad e^1 \approx 2.718282.$$

The denominator is

$$1.395612 + 2.718282 = 4.113894.$$

Therefore,

$$\frac{1.395612}{4.113894} \approx 0.339244,$$

$$\frac{2.718282}{4.113894} \approx 0.660756.$$

Thus,

row 2 values

$$a_2 = \boxed{\begin{bmatrix} 0.339244 & 0.660756 & 0 \end{bmatrix}}$$

Row 3

$$\text{softmax} \left(\begin{bmatrix} 1.000000 & 0.666667 & 1.333333 \end{bmatrix} \right).$$

The exponentials are approximately

$$e^1 = 2.718282,$$

$$e^{0.666667} = 1.947734,$$

$$e^{1.333333} = 3.793668.$$

The denominator is

$$2.718282 + 1.947734 + 3.793668 = 8.459684.$$

Therefore,

$$\frac{2.718282}{8.459684} \approx 0.321316,$$

$$\frac{1.947734}{8.459684} \approx 0.230233,$$

$$\frac{3.793668}{8.459684} \approx 0.448451.$$

Thus,

row 3 values

$$a_3 = \begin{bmatrix} 0.321316 & 0.230233 & 0.448451 \end{bmatrix}$$

The complete causal attention-weight matrix is

$$A = \text{softmax}(S_{\text{masked}}) = \begin{bmatrix} 1.000000 & 0 & 0 \\ 0.339244 & 0.660756 & 0 \\ 0.321316 & 0.230233 & 0.448451 \end{bmatrix}$$

Every row sums to 1, and all future-position weights are zero.

Multiply the attention weights by V

Now calculate

$$O = AV.$$

Therefore,

$$O = \begin{bmatrix} 1.000000 & 0 & 0 \\ 0.339244 & 0.660756 & 0 \\ 0.321316 & 0.230233 & 0.448451 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ 10 & 20 & 30 & 40 & 50 & 60 & 70 & 80 & 90 \\ 100 & 200 & 300 & 400 & 500 & 600 & 700 & 800 & 900 \end{bmatrix}$$

First output row

$$o_1 = 1.000000v_1 + 0v_2 + 0v_3.$$

Hence,

$$o_1 = [1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad 7 \quad 8 \quad 9]$$

Second output row

$$o_2 = 0.339244v_1 + 0.660756v_2.$$

For the first component,

$$o_{2,1} = 0.339244(1) + 0.660756(10) = 0.339244 + 6.607560 = 6.946804.$$

Therefore,

row 2 values

$o_2 = [6.946804 \quad 13.893608 \quad 20.840412 \quad 27.787216 \quad 34.734020 \quad 41.680824 \quad 48.6276]$
--

Third output row

$$o_3 = 0.321316v_1 + 0.230233v_2 + 0.448451v_3.$$

For the first component,

$$o_{3,1} = 0.321316(1) + 0.230233(10) + 0.448451(100).$$

Thus,

$$o_{3,1} = 0.321316 + 2.302330 + 44.845100 = 47.468746.$$

Therefore,

row 3 values

$o_3 = [47.468746 \quad 94.937492 \quad 142.406238 \quad 189.874984 \quad 237.343730 \quad 284.812476 \quad 3.]$
--

Now calculate

$$O = AV.$$

Therefore,

$$O = \begin{bmatrix} 1.000000 & 0 & 0 \\ 0.339244 & 0.660756 & 0 \\ 0.321316 & 0.230233 & 0.448451 \end{bmatrix} \begin{bmatrix} \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \end{matrix} \\ \begin{matrix} 10 & 20 & 30 & 40 & 50 & 60 & 70 & 80 & 90 \end{matrix} \\ \begin{matrix} 100 & 200 & 300 & 400 & 500 & 600 & 700 & 800 & 900 \end{matrix} \end{bmatrix}$$

v_1^1
 v_1^2
 v_1^3

$$\begin{aligned} \text{row 2 values} &= \gamma_1 \cdot [-- v_1^1 --] + \gamma_2 \cdot [-- v_1^2 --] + \gamma_3 \cdot [-- v_1^3 --] \\ &= 0.339244 \cdot [1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9] + \\ &\quad 0.660756 \cdot [10 \ 20 \ 30 \ 40 \ 50 \ 60 \ 70 \ 80 \ 90] + \\ &\quad 0 \cdot [100 \ 200 \ 300 \ 400 \ 500 \ 600 \ 700 \ 800 \ 900] \end{aligned}$$

Complete Calculation:

$$QK^T = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 3 & 1 \\ 3 & 2 & 4 \end{bmatrix}$$

Attention score matrix

$$\frac{QK^T}{\sqrt{9}} = \begin{bmatrix} 0.666667 & 0.333333 & 1.000000 \\ 0.333333 & 1.000000 & 0.333333 \\ 1.000000 & 0.666667 & 1.333333 \end{bmatrix}$$

Scaled attention score matrix

$$\text{Causal-masked scores} = \begin{bmatrix} 0.666667 & -\infty & -\infty \\ 0.333333 & 1.000000 & -\infty \\ 1.000000 & 0.666667 & 1.333333 \end{bmatrix}$$

Masked scaled attention score matrix

$$A = \begin{bmatrix} 1.000000 & 0 & 0 \\ 0.339244 & 0.660756 & 0 \\ 0.321316 & 0.230233 & 0.448451 \end{bmatrix}$$

Attention weight matrix

Finally,

Head1 (part of the Value output)

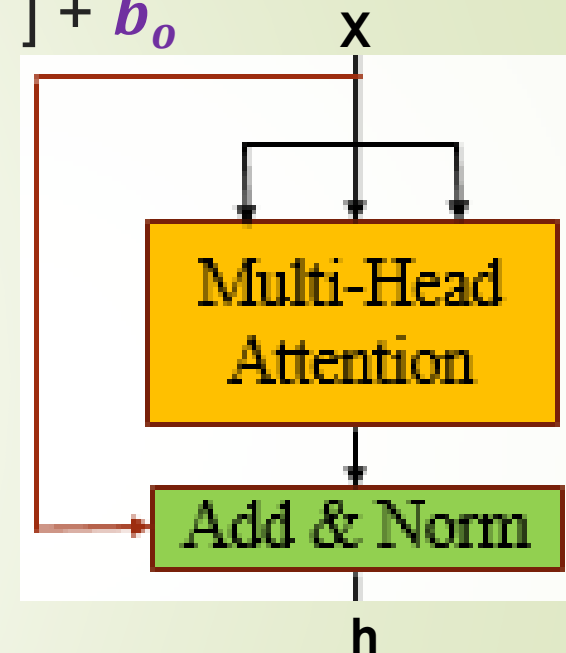
$$O = AV = \begin{bmatrix} 1.000000 & 2.000000 & 3.000000 & 4.000000 & 5.000000 & 6.000000 \\ 6.946804 & 13.893608 & 20.840412 & 27.787216 & 34.734020 & 41.680824 \\ 47.468746 & 94.937492 & 142.406238 & 189.874984 & 237.343730 & 284.812476 \end{bmatrix}$$

(3 x 9)

$$\begin{aligned}
 \text{Output} &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_o \\
 &= \begin{bmatrix} \text{Head1} & \text{Head2} & \text{Head3} & \text{Head4} \end{bmatrix} \begin{bmatrix} W_o \end{bmatrix} + b_o \\
 &\quad \underbrace{\begin{matrix} (3 \times 9) & (3 \times 9) & (3 \times 9) & (3 \times 9) \end{matrix}}_{(3 \times 36)}
 \end{aligned}$$

Output Weight Matrix
(36 x 36)

Dimension of Final Output
from multi-head attention is (3 x 36)



Hide the future: causal masking

A language model's job is next-token prediction: given everything so far, guess what comes next. If the token **"graded"** were allowed to peek at **"essays"** before it's even been generated, that would be cheating — attending to information it's not supposed to have yet. So the rule is strict: ****every token may only attend to itself and the tokens before it.****

Causal Mask, Lower Triangular

-  Allowed (attention score visible)
-  Masked (future tokens not visible)

	Token 1	Token 2	Token 3	Token 4	...	Token N
Token 1	score _{1,1}	masked	masked	masked	...	masked
Token 2	score _{2,1}	score _{2,2}	masked	masked	...	masked
Token 3	score _{3,1}	score _{3,2}	score _{3,3}	masked	...	masked
Token 4	score _{4,1}	score _{4,2}	score _{4,3}	score _{4,4}	...	masked
...	...	...	...	...	...	...
Token N	score _{N,1}	score _{N,2}	score _{N,3}	score _{N,4}	...	score _{N,N}

With a causal mask, each token can only attend to itself and previous token. Future tokens are masked.

5. Summary

Basic Building Blocks of a Transformer

Component	Function
Token Embeddings	Converts text into numerical vectors
Positional Encoding	Adds word order awareness
Self-Attention Layers	Helps each word attend to all others
Feedforward Layers	Learns complex patterns from attention outputs
Layer Normalization	Stabilizes and speeds up learning
Residual Connections	Combines past and current layers for better flow

Transformer is not LLM

From Transformer architecture to “foundation model”:

Everything above describes the *architecture* — but architecture alone doesn’t know how to chat, code, or translate. That comes from how the model is trained and adapted:

1. Pretraining — the model is trained on a massive, diverse body of text using a simple self-supervised objective: predict the next token. No manual labeling is required; the “labels” are just the next word in the text itself. Doing this at scale, across trillions of tokens, forces the model to implicitly learn grammar, facts, reasoning patterns, and even code.

2. Adaptation — the pretrained “base” model is then steered toward useful behavior through:

- **Prompting** — giving instructions and examples at request time, with no retraining at all (the model’s broad pretraining lets it generalize to new tasks just from a well-written prompt).

3. Fine-tuning — additional training on a smaller, task-specific dataset to specialize the model’s behavior.

4. Reinforcement learning from human feedback (RLHF) — human evaluators rank model outputs, and that feedback shapes the model toward helpful, honest, and safe behavior. This is largely how a raw text-completion model becomes something that behaves like a conversational assistant.

Before Transformer Blocks

Each token has a basic representation (embedding + position).

The

0.12 -0.03 0.45 0.08 ...

represents "The" + its position

player

-0.21 0.37 0.11 -0.05 ...

represents "player" + its position

was

0.06 -0.28 0.33 0.14 ...

represents "was" + its position

Transformer Blocks × N

Self-attention,
feed-forward,
(etc.)

After Transformer Blocks

Each token's representation is now context-aware (influenced by other tokens).

The

0.10 -0.12 0.28 0.07 ...

now includes context from "player" and "was"

player

-0.08 0.42 0.17 -0.03 ...

now includes context from "The" and "was"

was

0.03 -0.11 0.62 0.21 ...

now includes context from "The" and "player"

The numbers are just examples, but the key idea is:

After the Transformer blocks, each token's vector has been updated to include information from the other tokens.

The stock prediction demo example has a context window of 20.
ChatGPT-3 has a context window of 2048.

Why This Architecture Won

Why Transformers displaced RNNs as the default architecture for sequence data:

- **Parallelization:** Every word's attention can be computed simultaneously, making training dramatically faster on modern GPUs/TPUs compared to the strictly sequential nature of RNNs.
- **Long-range dependencies:** Any two words, no matter how far apart, are connected by a single attention computation — no information has to be relayed step-by-step through a chain of hidden states.
- **Scalability:** The architecture scales remarkably well — bigger models trained on more data reliably keep getting better, which is a large part of why it underlies GPT, BERT, T5, LLaMA, and virtually every major language model today.
- **Generality:** The same core idea (tokens attending to other tokens) has since been adapted well beyond text — to images (Vision Transformers), audio, protein structures, and more.

Summary

- Transformers replace recurrence with self-attention, letting every token directly attend to every other token. It works by comparing Queries to Keys to get relevance scores, scaling them, applying softmax, and using the result to weight a sum of Values.
- Multi-head attention runs several attention computations in parallel so the model can capture different types of relationships at once.
- Since attention alone has no sense of order, positional encoding is added to inject sequence position information.
- Feed-forward layers, residual connections, and layer normalization round out each Transformer block, and these blocks are stacked many times to build the full model.
- It is an architecture so powerful and so scalable that it now powers almost every state-of-the-art AI system.

On Attention Mechanism:

Instead of treating every word equally, attention allows a model to focus on the parts of the input that are most relevant to the current task. By using Queries, Keys, and Values, the model can calculate relationships between words and create context-aware representations.

Self-attention further allows every word in a sequence to interact with other words, while Multi-Head Attention enables the model to learn different types of relationships simultaneously.

Attention played a major role in the development of Transformer architectures, which now form the foundation of many modern AI systems. From machine translation and text summarization to question answering and conversational AI, attention has transformed the way machines process human language.

Ultimately, the key idea behind attention is simple:

Instead of trying to process everything equally, focus on what matters most.

In the stock value prediction demo:

$$d_k = \frac{D_{\text{MODEL}}}{N_{\text{HEAD}}} = \frac{64}{4} = 16$$

In ChatGPT-3

Context window of 2048

$D_{\text{model}} = 12288$

Number of Heads (in one Layer/Block) = 96

$d_k = 12288/96 = 128$

Number of Layers (N) = 96

#parameters = 175 billions

The End